Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

03
머신 러닝 입문하기

선형 회귀 하나로 "모델이 학습한다"는 말이 무슨 뜻인지 처음부터 끝까지 따라가 봅니다.

오늘 다루는 것

1. 선형 회귀와 자동 미분: 손실 함수와 경사하강법
2. 다중 선형 회귀
3. nn.Module과 클래스로 구현하기
4. 미니 배치와 데이터 로더
5. 벡터와 행렬 연산 복습하기

끝나고 나면 할 수 있는 것

1. "모델이 학습한다"가 실제로 무엇을 하는 것인지 설명할 수 있다.
2. 예측, 손실, 경사하강법으로 이어지는 흐름을 그림으로 그릴 수 있다.
3. 파이토치로 직선을 학습시키는 20줄짜리 코드를 직접 돌리고 읽을 수 있다.

2026년 10월 6일 화요일 14:30
발표자 설명, 코랩 실습, 퀴즈 돌아가며 풀기 순서로 진행합니다. 막힌 질문은 디스코드에 남겨 주세요.

03

머신 러닝 입문하기

"1시간 공부하면 2점, 2시간이면 4점, 3시간이면 6점. 그럼 4시간이면?" 사람은 바로 8점이라고 답합니다. 이 장에서는 **컴퓨터가 이 답을 스스로 찾아내게** 만듭니다. 일단 아무렇게나 예측해 보고, 얼마나 틀렸는지 재고, 덜 틀리는 쪽으로 조금씩 고치는 일을 반복합니다. 이 반복이 학습이고, 앞으로 배울 RNN, 트랜스포머, GPT도 똑같은 방식으로 학습합니다.

왜 배우나: 오늘의 학습 경로



03-01에서 이 경로를 끝까지 걷습니다. 그 뒤의 소챕터들은 같은 경로를 더 크게(다중 선형 회귀), 더 편하게(nn.Module), 더 빠르게(미니 배치) 걷는 도구입니다.

03-01 선형 회귀와 자동 미분

핵심 한 줄

데이터에 가장 잘 맞는 직선(W, b)을 찾기 위해, 손실로 틀린 정도를 재고 경사하강법으로 손실이 작아지는 쪽으로 조금씩 고친다.

왜 배우나

우리가 궁극적으로 하려는 일은 **잘 맞히는 예측 모델**을 만드는 것입니다. 그러려면 ① 예측을 어떤 식으로 할지(가설), ② 얼마나 틀렸는지(손실 함수), ③ 어떻게 덜 틀리게 고칠지(경사하강법)가 필요합니다. 이 세 가지가 머신 러닝 학습의 기본 뼈대이고, 이 소챕터에서 셋을 모두 만납니다.

1) 가설: 직선으로 예측해 보자

공부 시간 x로 점수 y를 맞히는 가장 단순한 방법은 **직선**입니다. 머신 러닝에서는 예측하는 식을 **가설(Hypothesis)**이라고 부르고, 직선은 이렇게 씁니다.

$$H(x) = Wx + b$$

말로 하면 "**W**(가중치, weight: 기울기)만큼 곱하고 **b**(편향, bias: 출발 높이)만큼 더한다"입니다. 중학교 때 배운 $y = ax + b$와 같습니다. **학습이란 결국 좋은 W와 b를 찾는 일**입니다.

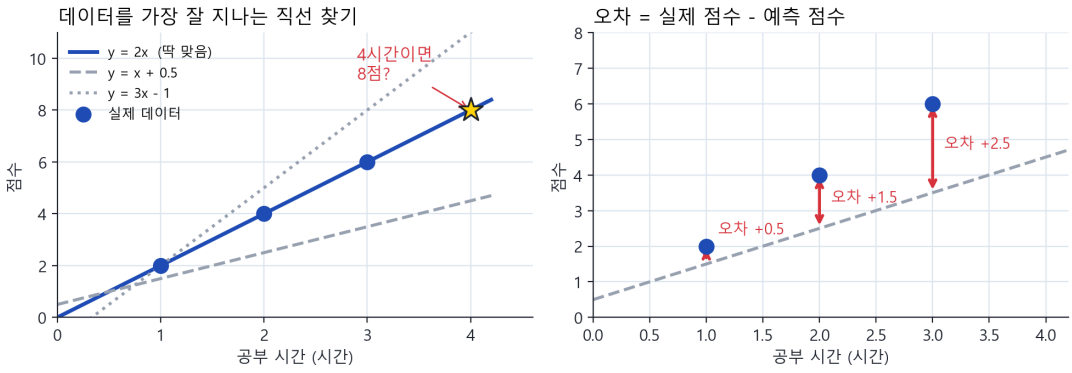


그림 1 왼쪽: 직선 후보는 무수히 많다. 데이터 점을 모두 지나는 초록 직선(y = 2x)이 정답. 오른쪽: 틀린 직선은 점마다 "오차"가 생긴다.

비유

> **택시 요금**
>
> 택시 요금 = **기본요금** + **거리당 요금** × 거리. 여기서 기본요금이 b, 거리당 요금이 W입니다. 영수증 몇 장(데이터)만 보고 "기본요금과 거리당 요금이 얼마일까?"를 거꾸로 알아내는 것이 선형 회귀입니다.

2) 손실 함수: 얼마나 틀렸는지 숫자로 재기

"이 직선이 더 잘 맞는 것 같아"라는 느낌으로는 컴퓨터가 일할 수 없습니다. 그래서 틀린 정도를 숫자 하나로 만듭니다.

- **오차** = 실제 점수 − 예측 점수, 즉 $y - H(x)$ (그림 오른쪽의 주황 화살표)
- 오차는 +도 −도 있어서 그냥 더하면 서로 지워집니다. 그래서 **제곱해서 모두 양수로** 만든 뒤 평균을 냅니다.
- 이것이 **평균 제곱 오차(MSE, Mean Squared Error)**이고, 선형 회귀의 **손실 함수(loss function)**입니다. 비용 함수(cost function)도 같은 뜻입니다.

$$\text{cost}(W, b) = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - H(x^{(i)})\right)^2$$

말로 하면: 데이터 n개 각각의 오차를 제곱해 모두 더한 뒤 n으로 나눈 평균. ∑(시그마)는 "모두 더하기"라는 뜻입니다.

그림 오른쪽 직선($y = x + 0.5$)의 오차는 +0.5, +1.5, +2.5이므로 손실은 다음과 같습니다. 정답 직선 $y = 2x$의 손실은 0입니다. **손실이 작을수록 좋은 직선**입니다.

$$\text{cost} = \frac{0.5^2 + 1.5^2 + 2.5^2}{3} = \frac{8.75}{3} \approx 2.92$$

3) 경사하강법: 손실이 가장 작은 곳 찾아가기

W를 바꿔 가며 손실을 그려 보면 아래처럼 **그릇 모양의 곡선**이 나옵니다. 우리가 원하는 것은 이 그릇의 **맨 밑바닥**(손실이 가장 작은 W)입니다.



그림 2 아무 W에서나 출발해도, 곡선의 경사(기울기)가 내려가는 쪽으로 한 걸음씩 옮기면 바닥(W = 2)에 도착한다.

방법은 이렇습니다. 지금 서 있는 곳의 **경사(기울기, gradient)**를 잽니다. 오른쪽이 오르막이면 왼쪽으로, 왼쪽이 오르막이면 오른쪽으로, 즉 **경사의 반대 방향**으로 한 걸음 옮깁니다. 이 과정을 반복하는 것이 **경사하강법(Gradient Descent)**입니다.

$$W \leftarrow W - \alpha \frac{\partial\, \text{cost}(W)}{\partial W}$$

새 W = 지금 W − 학습률(α) × 기울기. $\partial \text{cost}/\partial W$는 "W를 조금 바꿀 때 손실이 얼마나 바뀌나", 즉 그 지점의 경사입니다.

말로 하면 "기울기의 반대 방향으로, 학습률만큼의 보폭으로 이동한다"입니다. 바닥에서는 경사가 0이라서 더 움직이지 않습니다.

비유

> **산에서 길을 잃었을 때**
>
> 안개 낀 산에서 길을 잃었다면 **물소리를 찾아 따라가라**는 말이 있습니다. 물은 언제나 **경사를 따라 아래로** 흐르기 때문에, 물을 따라가면 결국 낮은 곳, 즉 마을에 닿습니다. 앞이 안 보여도 발밑의 경사만 알면 내려갈 수 있습니다.
>
> **지금 내 위치** = 현재 W
> **높이** = 손실
> **발밑의 경사** = 기울기(gradient)
> **마을** = 손실이 가장 작은 W

4) 학습률: 한 걸음의 크기

학습률(learning rate)은 한 번에 얼마나 크게 움직일지를 정합니다. 너무 작으면 하루 종일 걸리고, 너무 크면 바닥을 지나쳐 반대편 언덕으로 튕겨 나갑니다.

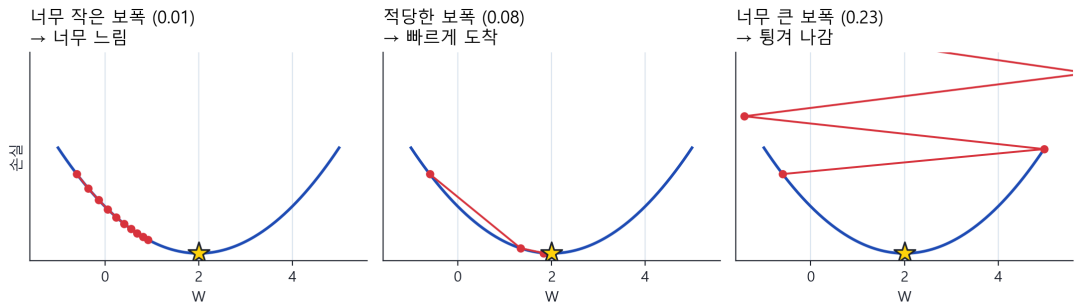


그림 3 같은 곡선, 다른 보폭. 오른쪽처럼 학습률이 너무 크면 손실이 오히려 커지며 발산한다.

5) 자동 미분: 기울기 계산은 파이토치가

경사하강법에는 매번 기울기(미분값)가 필요합니다. 다행히 직접 미분할 필요는 없습니다. 파이토치에서 `requires_grad=True`로 "이 값의 기울기가 필요해"라고 표시하고 `.backward()`를 부르면 자동으로 계산해 줍니다. 이것이 **자동 미분(Autograd)**입니다.

실습 1 자동 미분 맛보기

```python
import torch

# w = 2 에서 시작합니다. requires_grad=True : "이 값의 기울기를 계산해 줘"
w = torch.tensor(2.0, requires_grad=True)

y = 2 * w**2 + 5      # 식: y = 2w² + 5
y.backward()          # 미분(기울기 계산)을 파이토치에게 맡깁니다

# 손으로 풀면 dy/dw = 4w = 4 × 2 = 8
print("w = 2 일 때 기울기:", w.grad)
```

실행 결과

```
w = 2 일 때 기울기: tensor(8.)
```

실습 2 선형 회귀 처음부터 끝까지

```python
import torch
import torch.optim as optim

torch.manual_seed(1)  # 누가 실행해도 같은 결과가 나오게 고정

# 1) 데이터: 공부 시간(x) → 점수(y)
x_train = torch.FloatTensor([[1], [2], [3]])
y_train = torch.FloatTensor([[2], [4], [6]])

# 2) 처음에는 아무것도 모르는 상태: W = 0, b = 0
W = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)

# 3) 옵티마이저: 경사하강법(SGD), 한 걸음 크기(학습률) 0.01
optimizer = optim.SGD([W, b], lr=0.01)

for epoch in range(2001):
    hypothesis = x_train * W + b                    # 예측: H(x) = Wx + b
    cost = torch.mean((hypothesis - y_train) ** 2)  # 손실: 평균 제곱 오차(MSE)

    optimizer.zero_grad()  # 지난번 기울기 지우기
    cost.backward()        # 지금 위치의 기울기 계산
    optimizer.step()       # 기울기 반대 방향으로 W, b 한 걸음 이동

    if epoch % 400 == 0:
        print(f"Epoch {epoch:4d}  W: {W.item():.3f}  b: {b.item():.3f}  Cost: {cost.item():.6f}")

# 4시간 공부하면 몇 점일까?
print("4시간 공부 → 예상 점수:", round((4 * W + b).item(), 2))
```

실행 결과

```
Epoch    0  W: 0.187  b: 0.080  Cost: 18.666666
Epoch  400  W: 1.876  b: 0.281  Cost: 0.011366
Epoch  800  W: 1.953  b: 0.107  Cost: 0.001657
Epoch 1200  W: 1.982  b: 0.041  Cost: 0.000242
Epoch 1600  W: 1.993  b: 0.016  Cost: 0.000035
Epoch 2000  W: 1.997  b: 0.006  Cost: 0.000005
4시간 공부 → 예상 점수: 8.0
```

직접 해 보기 `lr=0.01`을 `0.1`, `0.3`으로 바꾸면 Cost가 어떻게 변하나요? 그림 「너무 큰 보폭」과 비교해 보세요.

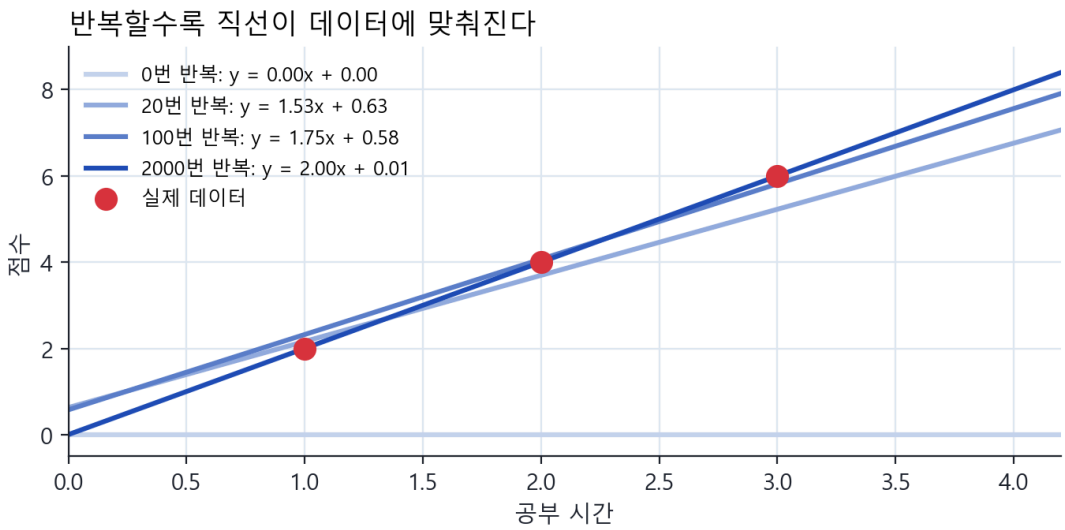


그림 4 실습 2를 반복할수록 직선이 데이터에 맞춰진다. 반복 횟수(에포크, epoch)가 늘수록 W는 2, b는 0에 가까워진다.

기억

학습 반복문의 세 줄, 이것만은 기억하기

`optimizer.zero_grad()` 지난 기울기 지우기 → `cost.backward()` 기울기 계산 → `optimizer.step()` 한 걸음 이동. 앞으로 GPT를 학습시킬 때도 이 세 줄은 그대로 나옵니다.

03-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 선형 회귀에서 학습이란 데이터에 잘 맞는 W와 b를 찾는 일이다. (O / X) 이름 _____

2 오차를 제곱하는 이유는 계산을 어렵게 만들기 위해서다. (O / X) 이름 _____

3 평균 제곱 오차(MSE)는 무엇을 재는 값인가요? 이름 _____

① 데이터의 개수 ② 예측이 정답과 얼마나 다른지 ③ 학습에 걸린 시간 ④ 직선의 기울기

4 학습률을 너무 크게 하면 어떤 일이 생길 수 있나요? 이름 _____

① 바닥을 지나쳐 손실이 오히려 커진다 ② 항상 더 정확해진다 ③ 데이터가 늘어난다

5 경사하강법은 기울기의 _____ 방향으로 W를 조금씩 옮긴다. 이름 _____

6 파이토치에서 기울기를 자동으로 계산하라고 부르는 함수는 `cost._____()` 이다. 이름 _____

7 "산에서 물소리를 따라가면 마을에 닿는다"는 비유로 경사하강법을 옆 사람에게 설명해 보세요. 물, 높이, 마을은 각각 무엇에 해당하나요? 이름 _____

원문: 03-01 선형 회귀와 자동 미분 — https://wikidocs.net/53560

03-02 다중 선형 회귀

핵심 한 줄

입력이 여러 개($x_1, x_2, x_3, \ldots$)여도 방법은 똑같다. 입력마다 가중치($w_1, w_2, w_3, \ldots$)를 하나씩 두고, 행렬 곱으로 한 번에 계산한다.

왜 배우나

현실의 예측은 단서가 하나뿐인 경우가 드뭅니다. 집값은 넓이, 역과의 거리, 층수로 정해지고, 문장의 감정은 수많은 단어로 정해집니다. 03-01에서 배운 "예측 → 손실 → 경사하강법"을 **입력이 여러 개인 경우로 넓히는 것**이 이 소챕터입니다. 여기서 쓰는 행렬 곱은 앞으로 모든 신경망의 기본 계산이 됩니다.

퀴즈 세 번의 점수로 기말 점수 맞히기

학생	퀴즈1 (x_1)	퀴즈2 (x_2)	퀴즈3 (x_3)	기말 (y)
1	73	80	75	152
2	93	88	93	185
3	89	91	90	180
4	96	98	100	196
5	73	66	70	142

가설은 직선에서 입력만 늘어난 모양입니다.

$$H(x) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

입력이 100개라면 식이 100칸이 됩니다. 그래서 학생들의 점수를 **표(행렬) X**, 가중치를 **세로줄(벡터) W**로 묶어 $H(X) = XW + b$ 한 줄로 씁니다(파이토치 코드로는 `X @ W + b`). (5명 × 퀴즈 3개) @ (퀴즈 3개 × 1) = (5명 × 1), 즉 **한 번에 5명 모두의 예측**이 나옵니다.

비유

> **성적 산출 비율**
>
> 학교 성적표에는 "중간고사 30% + 기말고사 40% + 수행평가 30%" 같은 반영 비율이 있습니다. 이 비율이 바로 가중치 w_1, w_2, w_3입니다. 다중 선형 회귀는 **비율표를 모르는 상태에서, 학생들의 점수와 최종 성적만 보고 비율을 거꾸로 알아내는 것**입니다.

실습 **행렬 곱으로 5명을 한 번에 예측**

```python
import torch
import torch.optim as optim

torch.manual_seed(1)

# 퀴즈 3개 점수(x) → 기말 점수(y). 5명의 학생 = 5행, 퀴즈 3개 = 3열
x_train = torch.FloatTensor([[73, 80, 75],
                             [93, 88, 93],
                             [89, 91, 90],
                             [96, 98, 100],
                             [73, 66, 70]])
y_train = torch.FloatTensor([[152], [185], [180], [196], [142]])

# 가중치는 퀴즈마다 하나씩 → 3행 1열
W = torch.zeros((3, 1), requires_grad=True)
b = torch.zeros(1, requires_grad=True)
optimizer = optim.SGD([W, b], lr=1e-5)  # 점수가 커서 학습률을 아주 작게

for epoch in range(1001):
    hypothesis = x_train.matmul(W) + b      # 행렬 곱 한 번으로 5명 모두 예측
    cost = torch.mean((hypothesis - y_train) ** 2)
    optimizer.zero_grad()
    cost.backward()
    optimizer.step()
    if epoch % 250 == 0:
        print(f"Epoch {epoch:4d}  Cost: {cost.item():.4f}")

print("예측:", hypothesis.squeeze().detach().round().tolist())
print("정답:", y_train.squeeze().tolist())
```

실행 결과

```
Epoch    0  Cost: 29661.8008
Epoch  250  Cost: 1.4659
Epoch  500  Cost: 1.3195
Epoch  750  Cost: 1.1915
Epoch 1000  Cost: 1.0794
예측: [152.0, 184.0, 181.0, 197.0, 140.0]
정답: [152.0, 185.0, 180.0, 196.0, 142.0]
```

직접 해 보기 `lr=1e-5`를 `1e-3`으로 키우면 어떻게 되나요? 입력 값(70~100)이 크면 왜 학습률을 작게 해야 할까요?

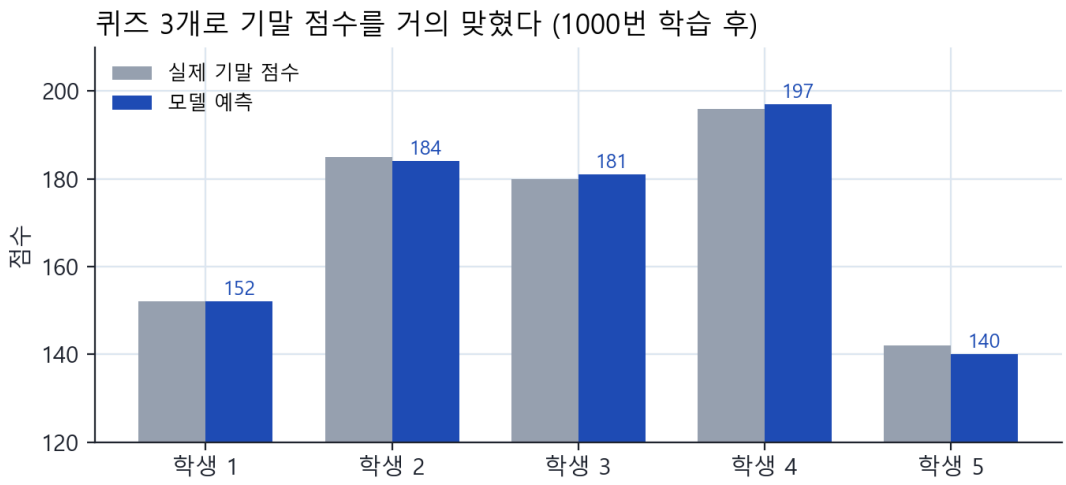


그림 5 실습 결과: 1000번 학습한 뒤 모델 예측(파랑)이 실제 기말 점수(회색)와 거의 같다.

03-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 다중 선형 회귀는 입력(x)이 여러 개인 선형 회귀다. (O / X) 이름 _____

2 입력이 3개면 가중치도 3개가 필요하다. (O / X) 이름 _____

3 실습에서 x_train의 모양(shape)은 무엇인가요? 이름 _____

① (3, 5) ② (5, 3) ③ (5, 1) ④ (1, 3)

4 입력을 행렬로 묶어 계산하는 가장 큰 이유는? 이름 _____

① 여러 데이터를 한 번에 빠르게 계산하려고 ② 그래프를 예쁘게 그리려고 ③ 손실을 크게 만들려고

5 H(X) = X _____ W + b — 빈칸에 들어갈 파이토치 연산 기호는? 이름 _____

6 성적 산출 비율 비유에서 "중간고사 30%"의 30%는 모델의 _____에 해당한다. 이름 _____

7 집값을 예측한다면 어떤 입력(x)들을 넣고 싶은지 세 가지 말해 보세요. 이름 _____

원문: 03-02 다중 선형 회귀 — https://wikidocs.net/54841

03-03 nn.Module과 클래스로 구현하기

핵심 한 줄

W와 b를 직접 만들고 MSE를 직접 쓰는 대신, 파이토치가 준비해 둔 nn.Linear, F.mse_loss, 그리고 모델 설계도인 nn.Module 클래스를 쓴다.

왜 배우나

03-01, 03-02에서는 W, b, 손실 식을 손으로 만들었습니다. 원리를 이해하기에는 좋지만, 신경망 층이 수십 개가 되면 손으로는 감당할 수 없습니다. 앞으로 볼 **모든 파이토치 모델(RNN, BERT, GPT…)은 nn.Module을 물려받은 클래스**로 만들어집니다. 오늘 이 모양을 익혀 두면 앞으로 나올 긴 모델 코드도 읽을 수 있습니다.

바뀌는 것 세 가지

지금까지 (직접 만들기)	이제부터 (파이토치 도구)	하는 일
`W = torch.zeros(1, requires_grad=True)` `b = torch.zeros(1, ...)`	`nn.Linear(1, 1)`	W와 b를 알아서 만들어 줌
`x_train * W + b`	`model(x_train)`	예측하기
`torch.mean((h - y) ** 2)`	`F.mse_loss(h, y)`	평균 제곱 오차 계산

클래스로 모델 설계도 만들기

클래스는 **설계도**입니다. `__init__`에서는 모델에 필요한 재료(층)를 준비하고, `forward`에서는 입력이 들어오면 어떤 순서로 계산할지(조리법)를 적습니다. `model(x)`라고 부르면 파이토치가 알아서 `forward`를 실행합니다.

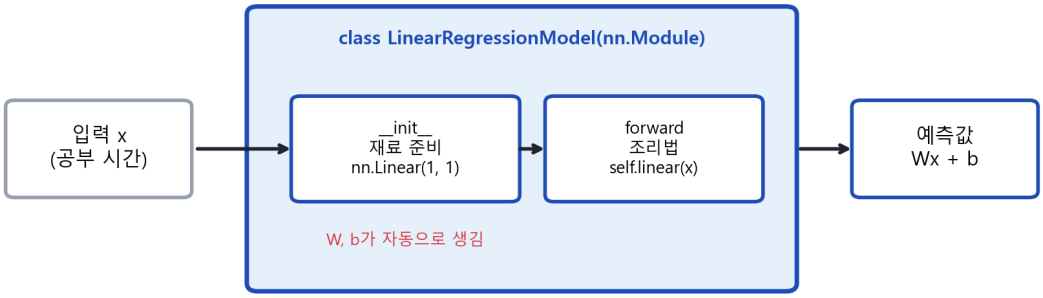


그림 6 nn.Module 클래스의 구조: 재료를 준비하는 __init__과 계산 순서를 정하는 forward

비유

밀키트

밀키트 상자에는 **손질된 재료**(__init__)와 **조리법 카드**(forward)가 함께 들어 있습니다. 상자를 열고 "요리해 줘"(model(x))라고만 하면 됩니다. nn.Module은 모든 밀키트가 따르는 **공통 상자 규격**이라서, 옵티마이저는 어떤 모델이든 `model.parameters()` 로 재료(W, b)를 꺼내 학습시킬 수 있습니다.

실습

03-01을 클래스로 다시 만들기

```
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(1)

x_train = torch.FloatTensor([[1], [2], [3]])
y_train = torch.FloatTensor([[2], [4], [6]])

# 우리만의 모델 '설계도'(클래스). nn.Module을 물려받습니다.
class LinearRegressionModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(1, 1)   # 입력 1개 → 출력 1개 (W, b가 자동으로
생김)

    def forward(self, x):               # 예측하는 방법
        return self.linear(x)

model = LinearRegressionModel()
print("학습 전 W, b:", [round(p.item(), 3) for p in model.parameters()])

optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
for epoch in range(2001):
    prediction = model(x_train)             # forward 자동 호출
    cost = F.mse_loss(prediction, y_train)  # 평균 제곱 오차를 한 줄로
    optimizer.zero_grad()
    cost.backward()
    optimizer.step()

print("학습 후 W, b:", [round(p.item(), 3) for p in model.parameters()])
print("4시간 공부 → 예상 점수:", round(model(torch.FloatTensor([[4.0]])).item
(), 2))
```

실행 결과

```
학습 전 W, b: [0.515, -0.441]
학습 후 W, b: [1.999, 0.001]
4시간 공부 → 예상 점수: 8.0
```

직접 해 보기 학습 전 W, b가 0이 아니라 0.515, -0.441인 이유는? (힌트: nn.Linear는 처음 값을 무작위로 정합니다. seed를 바꿔 보세요.)

확인 문제

03-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 nn.Linear(1, 1)을 쓰면 W와 b를 직접 만들지 않아도 된다. (O / X) 이름 _____

2 model(x)를 부르면 파이토치가 forward 함수를 실행한다. (O / X) 이름 _____

3 평균 제곱 오차를 한 줄로 계산해 주는 파이토치 함수는? 이름 _____

① F.mse_loss ② torch.zeros ③ nn.Module ④ optimizer.step

4 클래스 안에서 모델에 필요한 층(재료)을 준비하는 곳은? 이름 _____

① forward ② __init__ ③ backward

5 옵티마이저에 학습할 값들을 넘길 때 `model._____()` 를 쓴다. 이름 _____

6 우리가 만든 모델 클래스는 파이토치의 _____을(를) 물려받아 만든다. 이름 _____

7 밀키트 비유로 __init__과 forward의 차이를 설명해 보세요. 이름 _____

원문: 03-03 nn.Module과 클래스로 구현하기 — https://wikidocs.net/55409

03-04 미니 배치와 데이터 로더

핵심 한 줄

데이터가 많으면 한 번에 다 쓰지 않고 작은 묶음(미니 배치)으로 나눠 여러 번 학습한다. 나누고 섞는 일은 DataLoader가 해 준다.

왜 배우나

지금까지는 데이터가 3~5개라서 전부 한꺼번에 넣고 경사하강법을 했습니다. 하지만 자연어 처리에서는 문장이 수십만~수백만 개입니다. 전부를 한 번에 계산하면 너무 느리거나 메모리가 부족합니다. 그래서 **조금씩 나눠서 자주 고치는** 방식이 필요하고, 앞으로의 모든 실습은 이 DataLoader 방식으로 데이터를 넣습니다.

꼭 구분해야 할 세 단어

용어	뜻	예 (데이터 2,000개, 배치 크기 200)
배치 크기 (batch size)	한 묶음에 넣는 데이터 수	200개
이터레이션 (iteration)	W, b를 한 번 고치는 것 = 묶음 하나 처리	한 에포크에 2,000 ÷ 200 = 10번
에포크 (epoch)	전체 데이터를 한 바퀴 다 본 것	10번의 이터레이션 = 1 에포크



그림 7 학생 5명을 2명씩 나누면 묶음이 3개(2, 2, 1명)가 되고, 한 에포크에 이터레이션이 3번 일어난다. 에포크마다 순서를 섞는다.

전체를 한 번에 쓰는 방식(배치 경사하강법)은 안정적이지만 느리고, 미니 배치는 조금 흔들리지만 빠릅니다. 배치 크기는 보통 2의 거듭제곱(16, 32, 64…)을 씁니다.

비유

문제집 풀기

1,000쪽짜리 문제집을 다 푼 뒤에야 한 번 채점하면, 틀린 습관을 너무 늦게 고치게 됩니다. 20쪽씩 풀고 바로 채점하며 고치는 게 훨씬 빠르게 늡니다.

20쪽	= 미니 배치
한 번 채점하고 고치기	= 이터레이션
문제집 한 바퀴	= 에포크
매번 문제 순서를 섞는 것	= shuffle (순서를 외워서 맞히는 것을 막음)

실습

DataLoader로 2명씩 나눠 학습하기

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader

torch.manual_seed(1)

x_train = torch.FloatTensor([[73, 80, 75], [93, 88, 93], [89, 91, 90],
                             [96, 98, 100], [73, 66, 70]])
y_train = torch.FloatTensor([[152], [185], [180], [196], [142]])

dataset = TensorDataset(x_train, y_train)                    # x, y를 한 묶
음으로
dataloader = DataLoader(dataset, batch_size=2, shuffle=True)  # 2명씩, 매번
섞어서

model = nn.Linear(3, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-5)

for epoch in range(21):
    for batch_idx, (x_batch, y_batch) in enumerate(dataloader):
        cost = F.mse_loss(model(x_batch), y_batch)
        optimizer.zero_grad()
        cost.backward()
        optimizer.step()
        if epoch in (0, 20):
            print(f"Epoch {epoch:2d}  Batch {batch_idx + 1}/{len(dataloade
r)}"
                  f"  학생 수 {len(x_batch)}  Cost: {cost.item():.2f}")

print("[73, 80, 75] 학생의 예상 기말 점수:",
      round(model(torch.FloatTensor([[73, 80, 75]])).item(), 1))
```

실행 결과

```
Epoch  0  Batch 1/3  학생 수 2  Cost: 23071.78
Epoch  0  Batch 2/3  학생 수 2  Cost: 17581.36
Epoch  0  Batch 3/3  학생 수 1  Cost: 3703.55
Epoch 20  Batch 1/3  학생 수 2  Cost: 0.41
Epoch 20  Batch 2/3  학생 수 2  Cost: 0.19
Epoch 20  Batch 3/3  학생 수 1  Cost: 0.00
[73, 80, 75] 학생의 예상 기말 점수: 151.1
```

직접 해 보기 `batch_size`를 1, 5로 바꾸면 한 에포크의 Batch 개수는 몇 개가 되나요? 실제 정답(152)과 예측은 얼마나 차이 나나요?

확인 문제

03-04 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 에포크(epoch)는 전체 데이터를 한 바퀴 다 학습한 것을 말한다. (O / X) 이름 _____

2 미니 배치를 쓰면 전체 데이터를 한 번에 쓸 때보다 계산이 항상 더 느리다. (O / X) 이름 _____

3 데이터 1,000개, 배치 크기 100이면 한 에포크의 이터레이션은 몇 번인가요? 이름 _____

① 1번 ② 10번 ③ 100번 ④ 1,000번

4 DataLoader의 `shuffle=True` 는 무슨 일을 하나요? 이름 _____

① 데이터를 지운다 ② 에포크마다 데이터 순서를 섞는다 ③ 학습률을 바꾼다

5 한 묶음에 들어가는 데이터의 수를 _____(이)라고 한다. 이름 _____

6 실습에서 학생 5명을 2명씩 나누면 마지막 묶음에는 _____ 명이 들어간다. 이름 _____

7 문제집 비유로 배치, 이터레이션, 에포크를 설명해 보세요. 이름 _____

원문: 03-04 미니 배치와 데이터 로더 — https://wikidocs.net/55580

03-05 벡터와 행렬 연산 복습하기

핵심 한 줄

딥러닝의 데이터는 모두 텐서(숫자를 담는 상자)이고, 계산의 대부분은 행렬 곱(가로줄 × 세로줄을 곱해서 더하기)이다.

왜 배우나

03-02의 `X @ W` 처럼, 신경망은 결국 큰 행렬 곱을 반복하는 기계입니다. 실습하다 보면 가장 자주 만나는 에러가 **"모양(shape)이 안 맞는다"**입니다. 텐서의 차원과 행렬 곱의 규칙을 알면 이 에러를 스스로 고칠 수 있습니다. 자연어 처리에서는 문장 하나가 (단어 수 × 단어 벡터 크기) 행렬, 여러 문장이 3차원 텐서가 됩니다.

1) 텐서의 차원

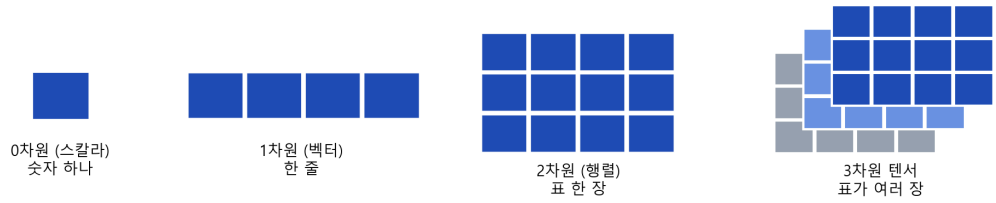


그림 8 차원이 하나 늘 때마다 "한 줄 → 표 → 표 묶음"처럼 숫자를 담는 방향이 하나씩 늘어난다.

비유

날씨 기록

오늘 기온 하나	= 0차원 (스칼라)
일주일 기온 한 줄	= 1차원 (벡터)
도시별, 요일별 기온표	= 2차원 (행렬)
그 표를 달마다 모은 묶음	= 3차원 텐서

2) 같은 자리끼리 곱하기 vs 행렬 곱

`A * B` 는 **같은 자리에 있는 숫자끼리** 곱합니다(요소별 곱, element-wise). `A @ B` (또는 `matmul`)는 **A의 가로줄과 B의 세로줄을 짝지어 곱한 뒤 더합니다**(행렬 곱). 행렬 곱이 되려면 **A의 열 수와 B의 행 수가 같아야** 합니다. (5×3) @ (3×1) → (5×1)



그림 9 행렬 곱의 첫 칸 계산: A의 첫 가로줄 [1, 3]과 B의 첫 세로줄 [5, 6]을 짝지어 곱하고 더하면 23

비유

장보기 영수증

사과 3개 × 1,000원 + 우유 2개 × 2,500원 = 8,000원. "개수 줄"과 "가격 줄"을 짝지어 곱하고 모두 더하는 계산이 바로 내적(행렬 곱의 한 칸)입니다. 손님이 여러 명이면 영수증 여러 장을 행렬 곱 한 번으로 계산할 수 있습니다.

실습

텐서 모양 확인과 두 가지 곱셈

```
import torch

s = torch.tensor(5.)                          # 0차원: 숫자 하나 (스칼라)
v = torch.tensor([1., 2., 3.])                # 1차원: 한 줄 (벡터)
m = torch.tensor([[1., 2.], [3., 4.]])        # 2차원: 표 (행렬)
print("차원:", s.dim(), v.dim(), m.dim())
print("모양:", s.shape, v.shape, m.shape)

A = torch.tensor([[1., 3.], [2., 4.]])
B = torch.tensor([[5., 7.], [6., 8.]])
print("같은 자리끼리 곱하기 A * B:\n", A * B)
print("행렬 곱 A @ B:\n", A @ B)

# 03-02 다중 선형 회귀의 모양 맞추기: (학생 5명 × 퀴즈 3개) @ (퀴즈 3개 × 1) =
(5 × 1)
X = torch.ones(5, 3)
W = torch.ones(3, 1)
print("X @ W 의 모양:", (X @ W).shape)
```

실행 결과

```
차원: 0 1 2
모양: torch.Size([]) torch.Size([3]) torch.Size([2, 2])
같은 자리끼리 곱하기 A * B:
 tensor([[ 5., 21.],
         [12., 32.]])
행렬 곱 A @ B:
 tensor([[23., 31.],
         [34., 46.]])
X @ W 의 모양: torch.Size([5, 1])
```

직접 해 보기 `W = torch.ones(2, 1)` 로 바꾸면 어떤 에러가 나나요? 에러 메시지에서 숫자 3과 2를 찾아보세요.

03-05 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 숫자 하나만 담은 텐서는 0차원(스칼라)이다. (O / X) 이름 _____

2 `A * B`와 `A @ B`는 항상 같은 결과를 낸다. (O / X) 이름 _____

3 엑셀 시트 한 장처럼 생긴 텐서는 몇 차원인가요? 이름 _____

① 0차원 ② 1차원 ③ 2차원 ④ 3차원

4 (5×3) 행렬과 곱할 수 있는 것은? 이름 _____

① (3×1) ② (5×1) ③ (2×3)

5 [1, 2, 3]과 [4, 5, 6]의 내적은 1×4 + 2×5 + 3×6 = _____ 이다. 이름 _____

6 (5×3) @ (3×1)의 결과 모양은 (_____ × _____) 이다. 이름 _____

7 장보기 영수증 비유로 행렬 곱(내적)이 어떤 계산인지 설명해 보세요. 이름 _____

원문: 03-05 벡터와 행렬 연산 복습하기 — https://wikidocs.net/217269

오늘의 요약

다시 보기



핵심 한 줄 모음

소챕터	핵심 한 줄
03-01	손실로 틀린 정도를 재고, 경사하강법으로 손실이 작아지는 쪽으로 W, b를 조금씩 고친다.
03-02	입력이 여러 개면 가중치도 여러 개. 행렬 곱 X @ W로 한 번에 계산한다.
03-03	nn.Linear, F.mse_loss, nn.Module 클래스로 같은 일을 더 짧고 정리된 코드로 한다.
03-04	데이터를 미니 배치로 나눠 자주 고친다. 배치 크기, 이터레이션, 에포크를 구분한다.
03-05	데이터는 텐서, 계산은 행렬 곱. 행렬 곱은 앞 행렬의 열 수와 뒤 행렬의 행 수가 같아야 한다.

기억

다음 시간 예고 (04장 로지스틱 회귀)

오늘은 "점수가 몇 점일까?"처럼 **숫자**를 맞혔습니다. 다음에는 "합격일까, 불합격일까?"처럼 **둘 중 하나**를 고르는 문제를 풉니다. 예측 → 손실 → 경사하강법의 뼈대는 그대로이고, 가설과 손실 함수만 바뀝니다.

정답과 해설

03-01 선형 회귀와 자동 미분

1. O — 직선은 W와 b로 정해지므로, 좋은 W와 b를 찾는 것이 곧 학습입니다.
2. X — +오차와 −오차가 서로 지워지지 않도록 모두 양수로 만들기 위해서입니다.
3. ② — MSE는 예측과 정답의 차이를 제곱해 평균 낸 값입니다.
4. ① — 보폭이 너무 크면 바닥을 지나쳐 반대편으로 튕겨 나가고 손실이 커질 수 있습니다.
5. 반대 — 오르막의 반대쪽, 즉 내려가는 쪽으로 이동합니다.
6. backward — `cost.backward()`가 W, b의 기울기를 계산합니다.
7. 예시 답: 물은 경사를 따라 아래로 흐르니 따라가면 마을에 닿는다. 경사하강법도 손실 곡선의 경사를 따라 내려가 손실이 가장 작은 곳을 찾는다. 물(내 위치) = 현재 W, 높이 = 손실, 마을 = 손실이 가장 작은 W

03-02 다중 선형 회귀

1. O
2. O — 입력마다 얼마나 중요한지를 나타내는 가중치가 하나씩 필요합니다.
3. ② — 학생 5명(행) × 퀴즈 3개(열)입니다.
4. ① — 행렬 곱 한 번으로 모든 데이터의 예측을 동시에 계산할 수 있습니다.
5. @ (또는 `matmul`)
6. 가중치(w)
7. 예시 답: 넓이, 역까지 거리, 층수, 지어진 연도, 방 개수 등. 정답은 없고, 집값에 영향을 줄 것 같은 단서면 모두 좋습니다.

03-03 nn.Module과 클래스로 구현하기

1. O — nn.Linear가 W와 b를 만들고 처음 값도 정해 줍니다.
2. O
3. ①
4. ② — __init__에서 재료(층)를 준비하고, forward에서 계산 순서를 정합니다.
5. parameters
6. nn.Module
7. 예시 답: __init__은 밀키트에 손질된 재료를 넣어 두는 것이고, forward는 그 재료로 요리하는 순서(조리법)다.

03-04 미니 배치와 데이터 로더

1. O
2. X — 미니 배치는 한 번의 계산량이 작아 더 빠르게, 더 자주 W와 b를 고칠 수 있습니다.
3. ② — 1,000 ÷ 100 = 10번입니다.
4. ② — 순서를 외워서 맞히는 것을 막고 학습을 고르게 만듭니다.
5. 배치 크기(batch size)

6. 1 — 2명, 2명, 1명으로 나뉩니다.
7. 예시 답: 20쪽씩 풀고(배치) 바로 채점해 고치고(이터레이션), 문제집을 한 바퀴 다 풀면 1 에포크다.

03-05 벡터와 행렬 연산 복습하기

1. O
2. X — *는 같은 자리끼리 곱하고, @는 가로줄 × 세로줄을 곱해 더합니다. 결과가 다릅니다.
3. ③
4. ① — 앞 행렬의 열 수(3)와 뒤 행렬의 행 수(3)가 같아야 합니다.
5. 32 — 4 + 10 + 18 = 32
6. 5 × 1
7. 예시 답: 물건 개수 줄과 가격 줄을 짝지어 곱한 뒤 모두 더하면 총액이 나온다. 이것이 내적이고, 행렬 곱의 한 칸이다.

용어 정리

용어	쉬운 뜻
가설 (Hypothesis)	예측하는 식. 선형 회귀에서는 H(x) = Wx + b
가중치 W / 편향 b	직선의 기울기 / 출발 높이. 학습으로 찾는 값
손실 함수, 비용 함수	예측이 얼마나 틀렸는지 숫자로 재는 식
평균 제곱 오차 (MSE)	오차를 제곱해서 평균 낸 값. 선형 회귀의 손실 함수
경사하강법	기울기의 반대 방향으로 조금씩 움직여 손실이 가장 작은 곳을 찾는 방법
학습률 (learning rate)	경사하강법의 한 걸음 크기
옵티마이저 (optimizer)	W, b를 실제로 고쳐 주는 도구. 예: optim.SGD
자동 미분 (Autograd)	backward()를 부르면 기울기를 자동으로 계산해 주는 파이토치 기능
에포크 / 배치 크기 / 이터레이션	전체 한 바퀴 / 한 묶음의 크기 / 한 번 고치기
텐서 (Tensor)	숫자를 담는 다차원 상자. 0차원 스칼라, 1차원 벡터, 2차원 행렬…
행렬 곱 (@, matmul)	가로줄 × 세로줄을 짝지어 곱하고 더하는 계산